

## AI ASSISTENT CODING - 11.4

**Baisa Vaishanvi**

**2303A52142**

**Batch - 41**

### **Task 1: Stack Implementation for Undo Operations (LIFO)**

#### **Scenario**

You are building a text editor where users can undo their recent actions (typing, deleting, formatting). Each action must be reversed in Last-In-First-Out (LIFO) order.

#### **Prompt:**

You are designing a text editor undo feature where actions must be reversed using the Last-In-First-Out (LIFO) principle. Generate a Python Stack class with methods `push(action)`, `pop()`, `peek()`, and `is_empty()`.

Provide a clean class skeleton with proper docstrings and comments.

Explain why a stack data structure is suitable for undo functionality.

Suggest and implement an alternative stack implementation using `collections.deque`. Include sample test cases that simulate undo operations and compare list-based vs deque-based stacks

#### **Code:**

C: > Users > Hello > 11\_1.py > ...

```
1
2 """Example module implementing a LIFO stack with undo simulation.
3
4 This script defines a `Stack` class supporting basic operations and
5 includes sample test code that mimics undoing a sequence of actions.
6
7 The bottom of the file contains a brief comparison between a list-based
8 implementation and one using `collections.deque`.
9 """
10
11 from collections import deque
12
13
14 class Stack:
15     """A simple last-in, first-out stack.
16
17     Attributes:
18         _items (list): internal container holding the stack elements.
19     """
20
21     def __init__(self):
22         """Initialize an empty stack."""
23         self._items = []
24
25     # -----
26     def push(self, action):
27         """Push an action onto the top of the stack.
28
```

C: > Users > Hello > 11\_1.py > ...

```
14 class Stack:
26     def push(self, action):
30         action: any object representing a user action or value.
31         """
32         self._items.append(action)
33
34     # -----
35     def pop(self):
36         """Remove and return the most recent action.
37
38         Returns:
39             The object that was added last.
40
41         Raises:
42             IndexError: if the stack is empty.
43         """
44         if self.is_empty():
45             raise IndexError("pop from an empty stack")
46         return self._items.pop()
47
48     # -----
49     def peek(self):
50         """Return the top element without removing it.
51
52         Raises:
53             IndexError: if the stack is empty.
54         """
55         if self.is_empty():
56             raise IndexError("peek from an empty stack")

```

C: > Users > Hello > 11\_1.py > ...

```
14 class Stack:
49     def peek(self):
54         """
55         if self.is_empty():
56             raise IndexError("peek from an empty stack")
57         return self._items[-1]
58
59     # -----
60     def is_empty(self):
61         """Return ``True`` if the stack contains no elements."""
62         return len(self._items) == 0
63
64
65     # -----
66     # sample tests simulating undo functionality
67     # -----
68     def _simulate_undo():
69         print("Creating history stack and performing actions...")
70         history = Stack()
71         actions = ["type 'hello'", "delete 'o'", "type '!', "save document"]
72
73         for act in actions:
74             print(f"Pushing: {act}")
75             history.push(act)
76
77         print("\nCurrent top action (peek):", history.peek())
78         print("\nUndoing actions:")
79         while not history.is_empty():
80             print("Undo:", history.pop())
```

```

82
83 if __name__ == "__main__":
84     _simulate_undo()
85
86
87 # -----
88 # Explanation: list vs deque
89 # -----
90 #
91 # The class above uses a Python list as the underlying container. Lists
92 # provide amortized O(1) performance for append() and pop() from the end,
93 # which makes them a natural choice for a stack. For most use cases the
94 # list implementation is simple and efficient.
95 #
96 # An alternative is to base the stack on ``collections.deque``. A deque is
97 # a double-ended queue optimized for fast appends and pops from both ends
98 # (O(1) guaranteed). The interface of a deque-based stack would be nearly
99 # identical; only the internal container changes:
100 #
101 #     self._items = deque()
102 #
103 # The performance advantage of ``deque`` becomes noticeable when the
104 # stack grows large or when operations on both ends are needed. Lists may
105 # occasionally incur a reallocation cost when growing, while ``deque``
106 # never does. However, for straightforward LIFO behavior either
107 # implementation is correct, and the list is often the simplest choice.

```

## Output:

```

PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

PS C:\Users\Hello\AppData\Local\Programs\Microsoft VS Code> & C:\Users\Hello\AppData\Local\Programs\Python\Python11-64\python.exe c:/Users/Hello/11_1.py
Creating history stack and performing actions...
Pushing: type 'hello'
Pushing: delete 'o'
Pushing: delete 'o'
Pushing: delete 'o'
Pushing: type '!'
Pushing: save document

Current top action (peek): save document

Undoing actions:
Undo: save document
Undo: type '!'
Undo: delete 'o'
Undo: type 'hello'
PS C:\Users\Hello\AppData\Local\Programs\Microsoft VS Code>

```

**Justification:**

Undo operations require reversing the most recent action first, which follows the LIFO principle. A stack naturally supports this behavior using push and pop operations. It ensures that the latest user action is undone before older actions. The structure is simple and efficient for editor operations. Using a list provides easy implementation and understanding. A deque-based stack improves performance for frequent operations.

**Task 2: Queue for Customer Service Requests (FIFO)****Scenario**

You are developing a customer support system where service requests must be handled in the order they arrive.

**Prompt:**

Generate a Python program to simulate a customer service request system using a **Queue (FIFO)**. First, implement a **Queue class using a Python list** with methods `enqueue()`, `dequeue()`, and `is_empty()`.

Then, **review the performance issues** of the list-based queue (especially dequeue operation). After that, **suggest and implement an optimized Queue using `collections.deque`**.

**Code:**

C: > Users > Hello > 11.2.py > ...

```
1  #Generate a Python program to simulate a customer service request system using a Queue
2  #Then, review the performance issues of the list-based queue (especially dequeue operation)
3  #After that, suggest and implement an optimized Queue using collections.deque.
4  #Finally, compare both implementations, explaining time complexity and why deque performs better.
5  from collections import deque
6  class ListQueue:
7      def __init__(self):
8          self.queue = []
9
10     def enqueue(self, item):
11         self.queue.append(item)
12
13     def dequeue(self):
14         if not self.is_empty():
15             return self.queue.pop(0) # This operation is O(n)
16         else:
17             raise IndexError("Dequeue from an empty queue")
18
19     def is_empty(self):
20         return len(self.queue) == 0
21 class DequeueQueue:
22     def __init__(self):
23         self.queue = deque()
24
25     def enqueue(self, item):
26         self.queue.append(item)
27
28     def dequeue(self):
29         if not self.is_empty():
```

```
27
28     def dequeue(self):
29         if not self.is_empty():
30             return self.queue.popleft() # This operation is O(1)
31         else:
32             raise IndexError("Dequeue from an empty queue")
33
34     def is_empty(self):
35         return len(self.queue) == 0
36 # Performance Comparison
37 import time
38 def performance_test(queue_class, num_operations):
39     queue = queue_class()
40     start_time = time.time()
41     for i in range(num_operations):
42         queue.enqueue(i)
43     for i in range(num_operations):
44         queue.dequeue()
45     end_time = time.time()
46     return end_time - start_time
47 num_operations = 100000
48 list_queue_time = performance_test(ListQueue, num_operations)
49 deque_queue_time = performance_test(DequeueQueue, num_operations)
50 print(f"List-based Queue Time: {list_queue_time:.4f} seconds")
51 print(f"Deque-based Queue Time: {deque_queue_time:.4f} seconds")
```

### Output:

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

PS C:\Users\Hello\AppData\Local\Programs\Microsoft VS Code> & C:\Users\Hello\
lo/11.2.py
List-based Queue Time: 13.3176 seconds
Deque-based Queue Time: 0.0193 seconds
PS C:\Users\Hello\AppData\Local\Programs\Microsoft VS Code> |
```

### Justification:

Customer requests must be handled fairly in the order they arrive. A queue follows the **FIFO principle**, ensuring first-come-first-served service. List-based queues are easy to implement but inefficient for large data.

Deque allows faster insertion and deletion from both ends. This reduces waiting time and improves system performance. Queues ensure scalability in real-time customer service systems.

## Task 3: Singly Linked List for Dynamic Playlist Management

### Scenario

You are designing a music playlist feature where songs can be added or removed dynamically while maintaining order.

### Prompt:

Generate a Python program to manage a music playlist using a Singly Linked List. Implement a Node class and a SinglyLinkedList class with the methods `insert_at_end(song)`, `delete_value(song)`, and `traverse()`.

Add inline comments explaining how pointers (next) are updated during insertion and deletion. Clearly highlight tricky parts, especially deleting the head node and traversing the list. Also, suggest and include edge case test scenarios such as an empty list, single-node list, and deletion of the first song.

### Code:

C: > Users > Hello > 11.3.py > ...

```
1  #Generate a Python program to manage a music playlist using a Singly Linked List.Implement a Node class
2  #Add inline comments explaining how pointers (next) are updated during insertion and deletion.
3  class Node:
4      def __init__(self, song):
5          self.song = song # Store the song name
6          self.next = None # Initialize the next pointer to None
7  class SinglyLinkedList:
8      def __init__(self):
9          self.head = None # Initialize the head of the list to None
10     def insert_at_end(self, song):
11         new_node = Node(song) # Create a new node with the given song
12         if not self.head: # If the list is empty, set the new node as the head
13             self.head = new_node
14             return
15         last_node = self.head # Start from the head of the list
16         while last_node.next: # Traverse to the end of the list
17             last_node = last_node.next
18         last_node.next = new_node # Update the next pointer of the last node to point to the new node
19     def delete_value(self, song):
20         current_node = self.head # Start from the head of the list
21         previous_node = None # Initialize previous node to None
22         while current_node: # Traverse through the list
23             if current_node.song == song: # If the song is found
24                 if previous_node: # If it's not the head node
25                     previous_node.next = current_node.next # Update previous node's next pointer
26                 else: # If it's the head node
27                     self.head = current_node.next # Update head to point to the next node
28                 return # Exit after deletion
29             previous_node = current_node # Move previous node to current node
30             current_node = current_node.next # Move to the next node in the list
```

C: > Users > Hello > 11.3.py > ...

```
7  class SinglyLinkedList:
31     def traverse(self):
32         songs = [] # Create a list to store songs for traversal output
33         current_node = self.head # Start from the head of the list
34         while current_node: # Traverse through the list until we reach the end (None)
35             songs.append(current_node.song) # Add the current song to the songs list
36             current_node = current_node.next # Move to the next node in the list
37         return songs # Return the list of songs in order
38 # Test cases
39 playlist = SinglyLinkedList()
40 # Edge case: Traversing an empty list
41 print("Playlist (should be empty):", playlist.traverse())
42 # Inserting songs into the playlist
43 playlist.insert_at_end("Song A")
44 playlist.insert_at_end("Song B")
45 playlist.insert_at_end("Song C")
46 print("Playlist after adding songs:", playlist.traverse())
47 # Edge case: Deleting the head node
48 playlist.delete_value("Song A")
49 print("Playlist after deleting the head node (Song A):", playlist.traverse())
50 # Edge case: Deleting a song that does not exist
51 playlist.delete_value("Song D")
52 print("Playlist after trying to delete a non-existent song (Song D):", playlist.traverse())
53 # Edge case: Deleting the last song
54 playlist.delete_value("Song C")
55 print("Playlist after deleting the last song (Song C):", playlist.traverse())
56 # Edge case: Deleting the only remaining song
57 playlist.delete_value("Song B")
58 print("Playlist after deleting the only remaining song (Song B):", playlist.traverse())
```



Output:

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS powershell
PS C:\Users\Hello\AppData\Local\Programs\Microsoft VS Code> & C:\Users\Hello\AppData\Local\Py
python.exe c:/Users/Hello/11.3.py
Playlist (should be empty): []
Playlist after adding songs: ['Song A', 'Song B', 'Song C']
Playlist after deleting the head node (Song A): ['Song B', 'Song C']
Playlist after trying to delete a non-existent song (Song D): ['Song B', 'Song C']
Playlist after deleting the last song (Song C): ['Song B']
Playlist after deleting the only remaining song (Song B): []
PS C:\Users\Hello\AppData\Local\Programs\Microsoft VS Code> █
```

### Justification:

A playlist requires dynamic addition and removal of songs. Linked lists allow efficient memory usage without shifting elements. Insertion and deletion are done by updating pointers. This maintains the order of songs in the playlist.

Edge cases like empty or single-node lists are handled easily. Linked lists are suitable for frequently changing data collections.

## Task 4: Binary Search Tree for Fast Record Lookup

### Scenario

You are building a student record system where quick searching by roll number is required.

### Prompt:

You are given a **partially implemented Node and Binary Search Tree (BST) class in Python** for a student record system where roll numbers are used as keys.

Complete the missing methods `insert(value)`, `search(value)`, and `inorder_traversal()`.

Add **meaningful docstrings** explaining the purpose of each method.

Ensure the BST maintains correct ordering properties during insertion.

Finally, **explain how BST search is more efficient than linear search**, including **best-case and worst-case time complexity**.

### Code:

C: > Users > Hello > 11.4.py > ...

```
1  #You are given a partially implemented Node and Binary Search Tree (BST) class
2  #Add meaningful docstrings explaining the purpose of each method.Ensure the BST
3  #Finally, explain how BST search is more efficient than linear search, includin
4  class Node:
5      """A node in the binary search tree.
6
7      Attributes:
8          roll_number (int): The key stored in the node.
9          left (Node|None): Left child (keys < roll_number).
10         right (Node|None): Right child (keys > roll_number).
11     """
12
13     def __init__(self, roll_number):
14         self.roll_number = roll_number
15         self.left = None
16         self.right = None
17
18
19     class BinarySearchTree:
20         """A simple Binary Search Tree (BST) for integer roll numbers.
21
22         Methods:
23             insert(roll_number): Insert a key preserving BST order.
24             search(roll_number): Return True if key exists, else False.
25             inorder_traversal(): Return a sorted list of keys.
26         """
27
28         def __init__(self):
29             self.root = None
```

```
19 class BinarySearchTree:
31     def insert(self, roll_number):
32         """Insert `roll_number` into the BST.
33
34         Duplicate values are ignored (no-op).
35         """
36         if self.root is None:
37             self.root = Node(roll_number)
38         else:
39             self._insert_recursive(self.root, roll_number)
40
41     def _insert_recursive(self, current_node, roll_number):
42         if roll_number < current_node.roll_number:
43             if current_node.left is None:
44                 current_node.left = Node(roll_number)
45             else:
46                 self._insert_recursive(current_node.left, roll_number)
47         elif roll_number > current_node.roll_number:
48             if current_node.right is None:
49                 current_node.right = Node(roll_number)
50             else:
51                 self._insert_recursive(current_node.right, roll_number)
52
53     def search(self, roll_number):
54         """Return True if `roll_number` exists in the tree, otherwise False."""
55         return self._search_recursive(self.root, roll_number)
56
57     def _search_recursive(self, current_node, roll_number):
58         if current_node is None:
```

```

19 class BinarySearchTree:
57     def _search_recursive(self, current_node, roll_number):
59         return False
60         if roll_number == current_node.roll_number:
61             return True
62         if roll_number < current_node.roll_number:
63             return self._search_recursive(current_node.left, roll_number)
64         return self._search_recursive(current_node.right, roll_number)
65
66     def inorder_traversal(self):
67         """Return a list of roll numbers from an inorder traversal (sorted order).
68         result = []
69         self._inorder_recursive(self.root, result)
70         return result
71
72     def _inorder_recursive(self, current_node, result):
73         if current_node is not None:
74             self._inorder_recursive(current_node.left, result)
75             result.append(current_node.roll_number)
76             self._inorder_recursive(current_node.right, result)
77
78
79     def _demo():
80         """Demo routine that builds a tree, shows traversal and search results.
81
82         Expected printed output is shown in the docstring below.
83         """
84
85         print("Building BST and inserting roll numbers...")
86         bst = BinarySearchTree()

```

C: > Users > Hello > 11.4.py > ...

```
76         self._inorder_recursive(current_node.right, result)
77
78
79     def _demo():
80         """Demo routine that builds a tree, shows traversal and search results.
81
82         Expected printed output is shown in the docstring below.
83         """
84
85         print("Building BST and inserting roll numbers...")
86         bst = BinarySearchTree()
87         values = [50, 30, 70, 20, 40, 60, 80]
88         for v in values:
89             print(f"Insert: {v}")
90             bst.insert(v)
91
92         print("\nInorder traversal (should be sorted):")
93         inorder = bst.inorder_traversal()
94         print(inorder)
95
96         # Search tests
97         tests = [60, 25]
98         for t in tests:
99             found = bst.search(t)
100             print(f"Search {t}: {found}")
101
102
103 if __name__ == "__main__":
104     _demo()
105
```

```
107 # -----
108 # AI-generated explanation: best-case vs worst-case performance
109 # -----
110 #
111 # Best case:
112 # - A search or insertion can run in  $O(1)$  if the tree contains only the
113 #   root and that node is the target (or we insert at root when tree empty).
114 # - More generally, for a balanced BST the time complexity for search,
115 #   insert, and delete is  $O(\log n)$  because each step eliminates half of
116 #   the remaining nodes on average.
117 #
118 # Worst case:
119 # - If the BST becomes unbalanced and degrades to a linked list (for
120 #   example, inserting already-sorted data into an empty BST), the height
121 #   becomes  $O(n)$ . In that case search and insert take  $O(n)$  time.
122 #
123 # Summary:
124 # - Balanced tree:  $O(\log n)$  average/worst (for self-balancing trees).
125 # - Plain BST: average  $O(\log n)$ , worst-case  $O(n)$ . Use self-balancing
126 #   trees (AVL, Red-Black) when worst-case guarantees are required.
```

## Output:

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS powershell + - [] ...
PS C:\Users\Hello\AppData\Local\Programs\Microsoft VS Code> & C:\Users\Hello\AppData\Local\Programs\Microsoft VS Code\python\pythoncore-3.14-64\python.exe c:/Users/Hello/11.4.py
Building BST and inserting roll numbers...
Insert: 50
Insert: 30
Insert: 70
Insert: 20
Insert: 40
Insert: 60
Insert: 80

Inorder traversal (should be sorted):
[20, 30, 40, 50, 60, 70, 80]
Search 60: True
Search 25: False
PS C:\Users\Hello\AppData\Local\Programs\Microsoft VS Code> |
```

## Justification:

Student records require fast searching by roll number. Binary Search Trees store data in a sorted hierarchical structure. This reduces search time compared to linear search.

Best-case search occurs in logarithmic time. Worst-case occurs when the tree becomes skewed. BSTs improve efficiency for large datasets.

## Task 5: Graph Traversal for Social Network Connections

### Scenario

You are modeling a social network, where users are connected to friends, and you want to explore connections.

### Prompt:

Design a Python program to model a social network using a graph represented as an adjacency list.

Implement Breadth-First Search (BFS) to find nearby or immediate connections between users.

Implement Depth-First Search (DFS) to explore deeper connection paths in the network.

Add inline comments explaining each step of BFS and DFS traversal.

Compare recursive DFS vs iterative DFS, and explain their advantages and limitations.

Finally, suggest practical real-world use cases where BFS is preferred over DFS and vice versa.

### Code:

C: > Users > Hello > 11.5.py > ...

```
1  #Design a Python program to model a social network using a graph represented as an adjacency list.
2  #Add inline comments explaining each step of BFS and DFS traversal. Compare recursive DFS with iterative DFS.
3  class Graph:
4      def __init__(self):
5          self.graph = {}
6
7      def add_edge(self, u, v):
8          if u not in self.graph:
9              self.graph[u] = []
10         if v not in self.graph:
11             self.graph[v] = []
12         self.graph[u].append(v)
13         self.graph[v].append(u)
14
15     def bfs(self, start):
16         visited = set()
17         queue = [start]
18         bfs_result = []
19
20         while queue:
21             vertex = queue.pop(0)
22             if vertex not in visited:
23                 visited.add(vertex)
24                 bfs_result.append(vertex)
25                 queue.extend([neighbor for neighbor in self.graph[vertex] if neighbor not in visited])
26
27         return bfs_result
28
29     def dfs_recursive(self, vertex, visited=None):
```

C: > Users > Hello > 11.5.py > ...

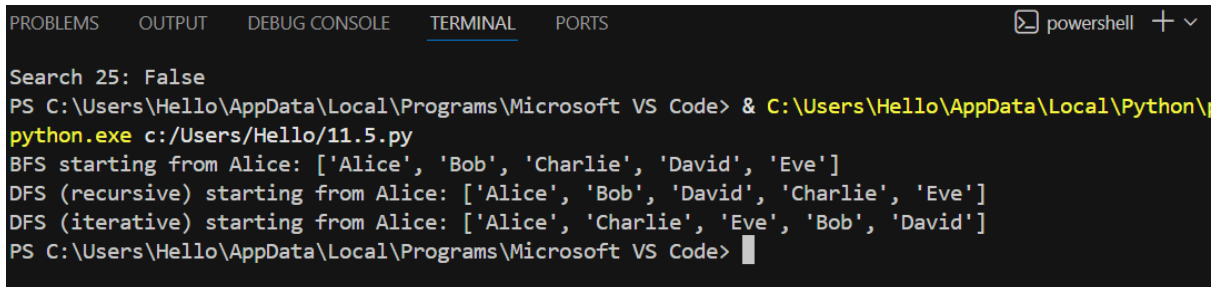
```
3  class Graph:
29     def dfs_recursive(self, vertex, visited=None):
30         if visited is None:
31             visited = set()
32         visited.add(vertex)
33         dfs_result = [vertex]
34
35         for neighbor in self.graph[vertex]:
36             if neighbor not in visited:
37                 dfs_result.extend(self.dfs_recursive(neighbor, visited))
38
39         return dfs_result
40
41     def dfs_iterative(self, start):
42         visited = set()
43         stack = [start]
44         dfs_result = []
45
46         while stack:
47             vertex = stack.pop()
48             if vertex not in visited:
49                 visited.add(vertex)
50                 dfs_result.append(vertex)
51                 stack.extend([neighbor for neighbor in self.graph[vertex] if neighbor not in visited])
52
53         return dfs_result
```

```

54 # Example usage
55 if __name__ == "__main__":
56     social_graph = Graph()
57     social_graph.add_edge("Alice", "Bob")
58     social_graph.add_edge("Alice", "Charlie")
59     social_graph.add_edge("Bob", "David")
60     social_graph.add_edge("Charlie", "Eve")
61
62     print("BFS starting from Alice:", social_graph.bfs("Alice"))
63     print("DFS (recursive) starting from Alice:", social_graph.dfs_recursive("Alice"))
64     print("DFS (iterative) starting from Alice:", social_graph.dfs_iterative("Alice"))

```

## Output:



```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
Search 25: False
PS C:\Users\Hello\AppData\Local\Programs\Microsoft VS Code> & C:\Users\Hello\AppData\Local\Python\python.exe c:/Users/Hello/11.5.py
BFS starting from Alice: ['Alice', 'Bob', 'Charlie', 'David', 'Eve']
DFS (recursive) starting from Alice: ['Alice', 'Bob', 'David', 'Charlie', 'Eve']
DFS (iterative) starting from Alice: ['Alice', 'Charlie', 'Eve', 'Bob', 'David']
PS C:\Users\Hello\AppData\Local\Programs\Microsoft VS Code>

```

## Justification:

Social networks consist of users connected in complex relationships. Graphs accurately represent these connections using nodes and edges. BFS helps find nearby or mutual connections quickly. DFS explores deep connection paths efficiently. Traversal helps analyze relationships and recommendations. Graphs support real-world network modeling effectively.